Model Wars: RLBot ML Competition

Participant Technical Specification and Submission Guide

Model Wars Tournament Committee

October 10, 2026

Abstract

This guide covers everything you need to build and submit a bot for the Model Wars Rocket League tournament: the observation and model contract, environment setup, submission rules, and the GitHub workflow. You train a neural policy that plays 1v1 Soccar through RLBot v5. Every submission is validated automatically by continuous integration (CI), including a cryptographic check on your trained weights.

Contents

1 Competition Overview and Model Contract

Your bot plays head-to-head 1v1 Rocket League matches against other participants' bots. It talks to the game through the official **RLBot v5** framework: on every callback it receives a physics-state observation and returns controller inputs. Callback frequency is variable, so the model sees the elapsed time between callbacks as an input.

1.1 Observation Contract (13 Dimensions)

The model receives exactly **13 finite `float32` values** per decision callback, laid out as in Table **??**. Car-relative displacements are projected onto the car's local axes (forward, right, up) and divided by the standard game constants listed in the table.

1.2 Neural Policy Architecture (26,181 Parameters)

To keep compute fair, every submission must use the exact **13D causal GRU architecture** below:

$$\mathbf{x} \in \mathbb{R}^{13} \xrightarrow{\text{Linear}(13\to64)} \mathbf{z}_1 \xrightarrow{\text{ReLU}} \mathbf{z}_2 \xrightarrow{\text{GRU}(64\to64)} \mathbf{h} \xrightarrow{\text{Linear}(64\to5)} \mathbf{y}$$

The output is $\mathbf{y} = [\mathbf{m}_{1:4},\ s]$, where:

- $\mathbf{m} \in \mathbb{R}^4$ are the **mode logits**, which select one of four discrete action primitives.
- $s \in [-1, 1]$ is the **continuous steering** value, produced by a $\tanh(\cdot)$ activation.
- The model has **26,181** trainable parameters in total.

1.3 Action Decoding

The selected mode determines the controller output as shown in Table **??**.

Idx	Feature	Normalization	Description
0	Ball Forward	$x/6000$ UU	Car-relative ball displacement, longitudinal axis.
1	Ball Right	$y/6000$ UU	Car-relative ball displacement, lateral axis.
2	Ball Up	$z/6000$ UU	Car-relative ball displacement, vertical axis.
3	Prediction Forward	$x_{\text{pred}}/6000$ UU	Slice of the 2-second RLBot ball trajectory prediction, longitudinal.
4	Prediction Right	$y_{\text{pred}}/6000$ UU	Same prediction slice, lateral.
5	Prediction Up	$z_{\text{pred}}/6000$ UU	Same prediction slice, vertical.
6	Ball Distance	$\|\mathbf{d}\|/6000$ UU	3D Euclidean distance from car to ball.
7	Car Speed	$\|\mathbf{v}\|/2300$ UU/s	Magnitude of the car's linear velocity.
8	Ball Present	Binary $\{0, 1\}$	Mask: 1 if the ball is present.
9	Prediction Valid	Binary $\{0, 1\}$	Mask: 1 if the trajectory slice is valid.
10	Horizon Offset	$\Delta t/2.0$	Prediction time offset: $(t_{\text{slice}} - t_{\text{now}})/2$.
11	Slice Offset	$\Delta t_{\text{first}}/(1/120)$	Offset relative to the first trajectory packet slice.
12	Callback Δt	$\Delta t/(1/60)$	Time since the previous callback (0 on the first tick).

Table 1: The 13-dimensional input observation vector.

Mode	Behavior	Throttle	Steering	Pitch	Jump
0	Neutral	0.0	0.0	0.0	False
1	Chase	1.0	Continuous, $s \in [-1, 1]$	0.0	False
2	Single Jump	0.0	0.0	0.0	True
3	Front Dodge (Flip)	0.0	0.0	-1.0	True

Table 2: Action mode table. Yaw, roll, boost, and handbrake default to 0 / False in the baseline.

2 Environment Setup and Local Testing

2.1 Prerequisites

Install the following before you start:

1. **Python 3.12 (64-bit)**, available on your system `PATH`.
2. **Rocket League**, installed through Epic Games or Steam.
3. **RLBot v5 Launcher**, from https://rlbot.org/v5/.

2.2 Initialize the Local Environment

Run the setup script from the repository root. It installs the pinned dependencies and creates the `.venv` virtual environment.

```
# Windows (PowerShell)
.\setup.ps1

# Or double-click SETUP.cmd
```

2.3 Run Local Verification

Check your environment and model tensors before you launch the game:

```
# Numerical and tensor integrity checks
.\.venv\Scripts\python.exe -I -u -B src/preflight.py

# Or double-click CHECK.cmd
```

Then test live gameplay: open the RLBot v5 launcher, click **Add File**, select `bot.toml`, and start a 1v1 match against any opponent bot. Keep notes on what you see, because you will need them for your technical report.

3 Submission Rules and Integrity Policy

Baseline Weights Are Rejected

Submissions that use the unmodified starter weights (`V13.pt`) or the default starter identifier (`shryssssss/v13`) are **automatically disqualified** by the CI validation system.

3.1 Submission Checklist

A valid submission must meet all of the following:

- **Trained weights.** The checkpoint SHA-256 must **not** equal the starter hash:
  `579fa26010140514d2e9b7f0e2871bee569c594b7da8648e31ee1489007cd5b7`
- **Custom identity.** `bot.toml` and `submission.json` must declare a distinct display name and an `author/bot` identifier.
- **Training provenance.** `submission.json` must contain non-empty experiment metadata: dataset version, training objective, seed, split methodology, and validation scores.

- **Technical report.** `docs/PARTICIPANT_RESULTS.md` must document your experiment process, training curves or metrics, and live GUI test results. Minimum length is 50 characters.
- **Security sandbox.** Code in `src/` must not import `socket`, `subprocess`, `requests`, or `urllib`, and must not use dynamic evaluation (`eval`, `exec`).

4 GitHub Submission Guide

Submissions go through private GitHub repositories, and GitHub Actions verifies them automatically.

Workflow Overview

1. Create a private repo → **2.** Invite the organizer → **3.** Initialize your bot identity → **4.** Train and seal → **5.** Push and check CI

4.1 Step 1: Create a Private Repository

1. Open the template repository: https://github.com/Model-Wars/Model-Wars-Base-Repository.
2. Click **Use this template**, then **Create a new repository**.
3. Name it `<team-name>-rlbot` (for example, `apex-rlbot`).
4. Set visibility to **Private**. A public repo exposes your model and code to every opponent.
5. Click **Create repository**.

4.2 Step 2: Add the Organizers as Collaborators

1. In your repo, go to **Settings** → **Collaborators**.
2. Click **Add people**.
3. Invite `@Model-Wars` (or the judge account the organizers designate) with **Read** or **Admin** access.

4.3 Step 3: Clone and Initialize Your Bot Identity

```
git clone https://github.com/<your-username>/<team-name>-rlbot.git
cd <team-name>-rlbot

# Set your bot name and agent ID in place
python tools/kit.py init --name "TeamApex" --agent-id "teamapex/striker"
```

4.4 Step 4: Train and Seal Your Checkpoint

Train your policy with imitation learning, reinforcement learning, or both. Then seal the resulting PyTorch weights into the repository:

```
# python tools/kit.py seal --checkpoint <model.pt> --provenance <
    provenance.json>
python tools/kit.py seal \
    --checkpoint <path_to_your_trained_model.pt> \
    --provenance docs/training_provenance.json
```

`seal` validates that all tensors are finite and in bounds, updates the SHA-256 pin in `src/runtime.py`, and records your training provenance in `submission.json`.

Do Not Seal the Starter Model

Pointing `-checkpoint` at the untouched starter `V13.pt` produces a hash match, and CI will reject the submission.

4.5 Step 5: Document Results and Push

Add your measured validation results to `docs/PARTICIPANT_RESULTS.md`, then commit and push:

```
git add .
git commit -m "Submit trained TeamApex policy"
git push origin main
```

5 Continuous Integration and Feedback

Every push to `main` triggers `.github/workflows/validate.yml`, which runs `tools/validate_submission.py`.

5.1 Reading the CI Result

- **[✓ PASS] Green check.** All requirements are met: the checkpoint is trained, the pinned hash matches, the tensor architecture is valid, and the documentation is present. Your submission is accepted.
- **[× FAIL] Red cross.** At least one check failed. Open the **Actions** tab on GitHub, read the build log, and find the failure message (missing provenance, unchanged starter hash, invalid tensor shape, and so on). Fix it locally, seal again, and push.

5.2 Common Validation Errors

Error Message	Fix
`Checkpoint matches unmodified starter weights!`	Replace `model/V13.pt` with your own trained weights using `kit.py seal`.
`PIN in src/runtime.py does not match actual SHA256`	Run `python tools/kit.py seal` to refresh the pin in `src/runtime.py`.
`submission.json missing!`	Run `python tools/kit.py init -name ... -agent-id ...` to create your bot metadata.
`PARTICIPANT_RESULTS.md missing or too brief`	Add detailed experiment notes and validation metrics to `docs/PARTICIPANT_RESULTS.md`.

Table 3: Troubleshooting common validation failures.

Model Wars Tournament Committee • All Rights Reserved